📌 Part 1 – Question Paper

Section A: 10-Mark Long Answer Questions

1. Explain the core capabilities, basic syntax, arithmetic expressions, null value behaviors, and display formatting options of the SQL SELECT statement with detailed code examples.
2. Discuss the WHERE clause in SQL comprehensively. Explain all comparison, logical, range (BETWEEN), list (IN), wildcard pattern matching (LIKE), and null handling operators with code examples.
3. Elaborate on SQL aggregate functions (COUNT, SUM, AVG, MIN, MAX), the GROUP BY clause, group filtering with HAVING, sorting aggregated data, and conditional aggregation using CASE statements.
4. Discuss the concept of relational joins in SQL. Detail INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, CROSS JOIN, and SELF JOIN with explicit syntax and multi-table examples.
5. Explain database Views and Integrity Constraints in SQL. Describe view creation, modification, updatability with WITH CHECK OPTION, and table constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK) along with commands to drop constraints.

Section B: 5-Mark Medium Answer Questions

6. Explain the definition of NULL in SQL, its impact on arithmetic expressions, and the syntax required to filter missing data using IS NULL and IS NOT NULL.
7. Detail the usage of column aliases, literal character strings, the concatenation operator (||), and the alternative quote (q) operator in SQL query design.
8. Explain row sorting using the ORDER BY clause, including ascending and descending order, multiple column sorting, alias sorting, and NULLS LAST handling.
9. Compare and contrast the WHERE clause and the HAVING clause in SQL using a comparative table and illustrative query examples.
10. Compare INNER JOIN and OUTER JOIN operations using a comparison table and SQL execution syntax.
11. Explain the WITH CHECK OPTION clause in SQL views, detailing its syntax, function, and importance in maintaining underlying table integrity during data modifications.
12. Differentiate between Primary Key and Foreign Key constraints using a comparison table and relational table definitions.
13. Compare ANSI standard SQL Statements with Oracle iSQL*Plus Commands using a comparative table and functional capabilities.
14. Explain how subqueries are utilized within the WHERE clause to dynamically filter table data, giving practical multi-table syntax examples.

Section C: 2-Mark Short Answer Questions

15. Define Projection and Selection as foundational capabilities of the SQL SELECT statement.
16. What is the purpose of the DISTINCT keyword in SQL queries? Provide its basic syntax.
17. Define the SQL LIKE operator and describe its two wildcard characters.
18. Differentiate between COUNT(*) and COUNT(column_name) in aggregate calculations.
19. Define a CROSS JOIN and state its mathematical relationship to joined table row counts.
20. Explain the concept and syntax of a SELF JOIN in SQL.
21. Define a View in SQL and state the SQL command used to delete a view.
22. How are integrity constraints dropped from existing database tables in SQL? State the command syntax.

📌 Part 2 – Answer Writing

Section A: 10-Mark Detailed Solutions

Q1. Explain the core capabilities, basic syntax, arithmetic expressions, null value behaviors, and display formatting options of the SQL SELECT statement with detailed code examples.

Introduction: The SQL SELECT statement is the primary Data Manipulation Language (DML) construct used to retrieve data from database tables. It allows database administrators and applications to query specific columns, filter rows, and combine records across multiple tables.

- **Projection Capability:**
  Projection is the operation of selecting specific columns from a table while omitting unwanted columns. It restricts the vertical dimension of the returned dataset so that only the necessary attributes are displayed.
  Syntax Example: SELECT first_name, last_name FROM employees;
- **Selection Capability:**
  Selection is the capability to restrict the rows returned by a query based on specified logical criteria. It acts as a horizontal filter across records in the table using the WHERE clause.
  Syntax Example: SELECT * FROM employees WHERE salary > 50000;
- **Joining Capability:**
  Joining combines rows from two or more related tables based on common key columns. It links attributes stored across normalized tables into a unified query output set.
  Syntax Example:
  SELECT employees.last_name, departments.department_name FROM employees INNER JOIN departments ON employees.department_id = departments.department_id;
- **Selecting All Columns (*):**
  Using an asterisk (*) in the SELECT clause retrieves every column defined in the target table schema in its original structural sequence. This is useful for quick exploratory data inspection.
  Syntax Example: SELECT * FROM employees;
- **Arithmetic Expressions:**
  Arithmetic operators (+, -, *, /) can be applied directly to numeric and date column values within the SELECT list to generate computed output columns. Calculations execute dynamically without altering stored table values.
  Syntax Example: SELECT last_name, salary, salary + 300 FROM employees;
- **Operator Precedence in Calculations:**
  SQL follows standard mathematical operator precedence, performing multiplication (*) and division (/) before addition (+) and subtraction (-). Parentheses () override standard precedence to force desired evaluation orders.
  Syntax Example: SELECT last_name, salary, 12 * (salary + 100) FROM employees;
- **Null Value Behavior in Arithmetic:**
  A NULL represents an unassigned, unknown, or inapplicable value, distinct from zero or a blank space. Any arithmetic expression containing a NULL operand automatically evaluates to NULL.
  Syntax Example: SELECT last_name, 12 * salary * commission_pct FROM employees;
- **Column Aliases (AS Keyword):**
  Aliases rename query output column headings to improve report readability or describe calculated expressions. Double quotation marks are required if the alias contains spaces,

special characters, or requires preserved case sensitivity.
Syntax Example:
SELECT last_name AS "Employee Name", salary * 12 "Annual Salary" FROM employees;

- **Duplicate Row Elimination (DISTINCT):**
  By default, SQL displays all rows evaluated by the query, including duplicate values across selected columns. The DISTINCT keyword forces the database engine to suppress duplicates and return only unique values.
  Syntax Example: SELECT DISTINCT department_id FROM employees;
- **Literal Character Strings and Concatenation (||):**
  A literal character string is freeform text included directly in the SELECT list, enclosed in single quotation marks. The concatenation operator (||) merges text literals and column values into single output character fields.
  Syntax Example:
  SELECT last_name || ' is a ' || job_id AS "Employee Details" FROM employees;

Q2. Discuss the WHERE clause in SQL comprehensively. Explain all comparison, logical, range (BETWEEN), list (IN), wildcard pattern matching (LIKE), and null handling operators with code examples.

Introduction: The WHERE clause restricts query output by establishing conditional criteria that every row must satisfy to be included in the result set. It immediately follows the FROM clause in standard SELECT statement execution order.

- **Basic Filtering Mechanism:**
  The WHERE clause evaluates logical conditions for each row in the source table. Rows evaluating to TRUE are retained, while rows evaluating to FALSE or NULL are excluded.
  Syntax Example: SELECT * FROM employees WHERE department_id = 90;
- **Standard Comparison Operators:**
  SQL supports basic relational comparison operators, including equal (=), not equal (<> or !=), greater than (>), less than (<), greater than or equal (>=), and less than or equal (<=). These evaluate numeric, text, and date data types.
  Syntax Example: SELECT * FROM employees WHERE salary >= 50000;
- **Logical AND Operator:**
  The AND logical operator combines two or more conditions, requiring every individual condition to evaluate to TRUE for a row to be retrieved. It restricts output to the strict intersection of conditions.
  Syntax Example: SELECT * FROM employees WHERE department = 'HR' AND salary > 50000;
- **Logical OR Operator:**
  The OR operator combines multiple conditions, returning a row if at least one of the specified criteria evaluates to TRUE. It expands query output across alternative conditional paths.
  Syntax Example:
  SELECT * FROM employees WHERE department = 'HR' OR department = 'Finance';
- **Logical NOT Operator:**
  The NOT operator negates a logical condition, returning records where the specified expression is FALSE. It is commonly prefixed to range, list, or pattern operators like NOT IN or NOT LIKE.
  Syntax Example: SELECT * FROM employees WHERE NOT department = 'HR';

- **Inclusive Range Filtering (BETWEEN ... AND ...):**
  The BETWEEN operator filters data within an inclusive specified range, matching values greater than or equal to the lower bound and less than or equal to the upper bound.
  Syntax Example: SELECT * FROM employees WHERE salary BETWEEN 40000 AND 60000;
- **Discrete Value List Matching (IN):**
  The IN operator tests whether a column value matches any discrete value within a parenthesized list. It serves as a concise, efficient alternative to multiple chained OR conditions.
  Syntax Example:
  SELECT * FROM employees WHERE department IN ('Sales', 'Marketing', 'Finance');
- **Pattern Matching with Wildcards (LIKE):**
  The LIKE operator performs search pattern matching on text fields using two primary wildcard characters: % (matches zero, one, or multiple characters) and _ (matches exactly one character).
  Syntax Example: SELECT * FROM employees WHERE first_name LIKE 'A%';
- **Null Value Testing (IS NULL / IS NOT NULL):**
  Because NULL represents missing data, standard comparison operators (=, !=) cannot evaluate nulls. Testing requires the explicit IS NULL or IS NOT NULL operators.
  Syntax Example: SELECT * FROM employees WHERE manager_id IS NULL;
- **Rules of Precedence for Combined Logical Operators:**
  In complex filtering expressions, comparison operators execute first, followed by NOT, then AND, and finally OR. Parentheses () must be used to force custom precedence or ensure logical clarity.
  Syntax Example:
  SELECT * FROM employees WHERE (department = 'HR' OR department = 'Finance') AND salary > 50000;

Q3. Elaborate on SQL aggregate functions (COUNT, SUM, AVG, MIN, MAX), the GROUP BY clause, group filtering with HAVING, sorting aggregated data, and conditional aggregation using CASE statements.

Introduction: Aggregate functions compute a single summary result from a collection of input column values across multiple rows. They are fundamental for analytical summaries, reporting, and categorical group evaluations.

- **COUNT() Function:**
  COUNT() returns the total number of rows matching query criteria. COUNT(*) counts all rows including duplicates and nulls, whereas COUNT(column) ignores NULL values.
  Syntax Example: SELECT COUNT(*) AS total_employees FROM employees;
- **SUM() Function:**
  SUM() calculates the cumulative numeric total of all non-null values in a specified column. It cannot be applied to character string or date fields.
  Syntax Example: SELECT SUM(salary) AS total_salaries FROM employees;
- **AVG() Function:**
  AVG() calculates the mathematical arithmetic mean of numeric column values, automatically ignoring NULL records in both numerator sum and denominator count.
  Syntax Example: SELECT AVG(salary) AS average_salary FROM employees;

- **MIN() and MAX() Functions:**
  MIN() and MAX() identify the lowest and highest values in a dataset, respectively. Unlike SUM() and AVG(), both functions operate on numeric, character string, and date data types.
  Syntax: SELECT MAX(salary) AS highest_sal, MIN(salary) AS lowest_sal FROM employees;
- **GROUP BY Clause Mechanics:**
  The GROUP BY clause divides query source data into distinct subsets based on identical values in specified columns. Aggregate functions then compute separate summary figures for each group.
  Syntax Example: SELECT department, COUNT(*) AS num_employees FROM employees GROUP BY department;
- **Grouping across Multiple Columns:**
  Grouping can extend across multiple hierarchical columns to create sub-groups. Rows are combined only when values across all specified GROUP BY columns match identically.
  Syntax Example:
  SELECT department, job_title, COUNT(*) FROM employees GROUP BY department, job_title;
- **Group Filtering using the HAVING Clause:**
  The HAVING clause filters aggregated data groups *after* GROUP BY summaries are computed. It evaluates aggregate expression conditions that cannot be evaluated within the WHERE clause.
  Syntax Example:
  SELECT department_id, COUNT(*) FROM employees GROUP BY department_id HAVING COUNT(*) > 10;
- **Sorting Aggregated Results (ORDER BY with Aggregates):**
  Summarized outputs can be sorted by placing aggregate function calls or aggregate column aliases inside the trailing ORDER BY clause.
  Syntax Example:
  SELECT department, AVG(salary) AS avg_sal FROM employees GROUP BY department ORDER BY avg_sal DESC;
- **Combining Joins with Aggregations:**
  Aggregate functions and GROUP BY clauses can process multi-table join result sets to generate comparative organizational reports.
  Syntax Example:
  SELECT departments.department_name, SUM(employees.salary) AS total_sal FROM employees INNER JOIN departments ON employees.department_id = departments.department_id GROUP BY departments.department_name;
- **Conditional Aggregation using CASE Expressions:**
  Placing a conditional CASE expression inside an aggregate function allows for selective row processing within groups without requiring multiple distinct queries.
  Syntax Example:
  SELECT department, SUM(CASE WHEN gender = 'Male' THEN 1 ELSE 0 END) AS male_count, SUM(CASE WHEN gender = 'Female' THEN 1 ELSE 0 END) AS female_count FROM employees GROUP BY department;

Q4. Discuss the concept of relational joins in SQL. Detail INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, CROSS JOIN, and SELF JOIN with explicit syntax and multi-table examples.

Introduction: A JOIN clause combines records from two or more database tables based on logical relationships between related key columns. Joins allow normalized database tables to be queried as a unified dataset.

- **Relational Join Overview:**
  Database normalization stores separate entity types in dedicated tables. Joins reconstruct entity relationships dynamically during query execution by evaluating matching criteria across primary key and foreign key fields.
- **INNER JOIN Mechanics:**
  An INNER JOIN returns rows only when there is an exact matching value in the join key columns of both joined tables. Unmatched rows from either table are completely omitted from the output set.
  Syntax Example:
  SELECT employees.employee_id, departments.department_name FROM employees INNER JOIN departments ON employees.department_id = departments.department_id;
- **LEFT (OUTER) JOIN Operation:**
  A LEFT JOIN returns all records from the left-hand table and matched records from the right-hand table. If no match exists in the right table, NULL values are returned for right-table columns.
  Syntax Example:
  SELECT employees.last_name, departments.department_name FROM employees LEFT JOIN departments ON employees.department_id = departments.department_id;
- **RIGHT (OUTER) JOIN Operation:**
  A RIGHT JOIN returns all records from the right-hand table and matching records from the left-hand table. Unmatched records from the left table populate as NULL values.
  Syntax Example:
  SELECT employees.last_name, departments.department_name FROM employees RIGHT JOIN departments ON employees.department_id = departments.department_id;
- **FULL (OUTER) JOIN Operation:**
  A FULL OUTER JOIN combines the outputs of both LEFT and RIGHT joins. It returns all records from both tables, filling unmatched attributes with NULL values on either side.
  Syntax Example:
  SELECT employees.last_name, departments.department_name FROM employees FULL OUTER JOIN departments ON employees.department_id = departments.department_id;
- **CROSS JOIN (Cartesian Product):**
  A CROSS JOIN pairs every row from the first table with every row in the second table, producing a Cartesian product. If Table A has \(M\) rows and Table B has \(N\) rows, the result set contains \(M \times N\) rows.
  Syntax Example:
  SELECT employees.last_name, projects.project_name FROM employees CROSS JOIN projects;
- **SELF JOIN Concept:**
  A SELF JOIN joins a table to itself as if it were two distinct tables. It relies on table aliases to differentiate the two roles of the same table, commonly used to query hierarchical data like employee-manager pairs.

Syntax Example:
SELECT E1.last_name AS employee, E2.last_name AS manager FROM employees E1, employees E2 WHERE E1.manager_id = E2.employee_id;

- **Multi-Table Join Queries:**
  Queries can chain multiple JOIN clauses to link three or more tables sequentially in a single logical transaction.
  Syntax Example:
  SELECT projects.project_name, COUNT(employee_projects.employee_id) FROM projects INNER JOIN employee_projects ON projects.project_id = employee_projects.project_id GROUP BY projects.project_name;
- **Filtering Outer Joins for Missing Matches:**
  Combining a LEFT JOIN with a WHERE ... IS NULL or HAVING ... = 0 clause identifies orphan records or unlinked parent entities, such as departments that have no assigned employees.
  Syntax Example:
  SELECT departments.department_name FROM departments LEFT JOIN employees ON departments.department_id = employees.department_id GROUP BY departments.department_name HAVING COUNT(employees.employee_id) = 0;
- **Join Execution Best Practices:**
  Always qualify joined column names with table names or short table aliases to avoid column ambiguity errors and improve query performance and execution clarity.

Q5. Explain database Views and Integrity Constraints in SQL. Describe view creation, modification, updatability with WITH CHECK OPTION, and table constraints (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK) along with commands to drop constraints.

Introduction: Views and integrity constraints enforce data abstraction, security, and schema validation in relational databases. Views present dynamic virtual tables, while constraints enforce mandatory data consistency rules on table columns.

- **Definition and Purpose of Views:**
  A View is a virtual database table defined by an underlying SELECT query result set. Views do not physically store data records; instead, they pull data dynamically from base tables upon execution.
- **Creating Views (CREATE VIEW):**
  Views are created using the CREATE VIEW statement followed by an assigned view name and the defining query.
  Syntax Example:
  CREATE VIEW details_view AS SELECT name, address FROM student_details WHERE student_id < 5;
- **Modifying Views (CREATE OR REPLACE VIEW):**
  To alter a view definition without dropping existing privileges, use CREATE OR REPLACE VIEW.
  Syntax Example:
  CREATE OR REPLACE VIEW marks_view AS SELECT student_details.name, student_marks.marks FROM student_details, student_marks WHERE student_details.name = student_marks.name;
- **Deleting Views (DROP VIEW):**
  A view can be permanently removed from the database schema using the DROP VIEW command without affecting base table data.

Syntax Example:
DROP VIEW marks_view;

- **Updatable Views (INSERT / DELETE operations):**
  Data modifications (INSERT, UPDATE, DELETE) executed against simple single-table views pass directly through to update the underlying base table.
  Syntax Example:
  INSERT INTO details_view (name, address) VALUES ('Suresh', 'Gurgaon');
- **Enforcing Rules with WITH CHECK OPTION:**
  The WITH CHECK OPTION clause added to a view definition prevents INSERT or UPDATE operations through the view that would produce rows failing to satisfy the view's WHERE clause condition.
- **Integrity Constraints Overview:**
  Constraints enforce rules on table columns to restrict allowed data entries, maintaining database accuracy, consistency, and reliability. They can be applied at the column level or table level.
- **Core Integrity Constraint Types:**
  - **NOT NULL**: Guarantees that a column cannot store NULL values.
  - **UNIQUE**: Ensures all non-null values in a column are distinct.
  - **PRIMARY KEY**: Uniquely identifies each record in a table; strictly forbids nulls.
  - **FOREIGN KEY**: Enforces referential integrity by linking a column to a primary key in another table.
  - **CHECK**: Verifies that column values satisfy specific conditional expressions.
- **Dropping Table Constraints (DROP CONSTRAINT):**
  Constraints can be removed from an existing table structure using the ALTER TABLE ... DROP CONSTRAINT statement.
  Syntax Example:
  ALTER TABLE employees DROP CONSTRAINT employees_pk;
- **System Shortcut Commands for Constraints:**
  Certain database engines support direct shortcuts for common constraints, such as ALTER TABLE employees DROP PRIMARY KEY;.

Section B: 5-Mark Medium

Q6. Explain the definition of NULL in SQL, its impact on arithmetic expressions, and the syntax required to filter missing data using IS NULL and IS NOT NULL.

Introduction: In SQL, a NULL value represents data that is missing, unassigned, unknown, or inapplicable. It is fundamentally distinct from a numeric zero (0) or a blank text string.

- **Definition of Null Values:**
  A NULL indicates the complete absence of a stored value in a record field. It signifies unknown information that cannot be evaluated using standard equivalency operations.
- **Null Values in Arithmetic Calculations:**
  Any mathematical arithmetic operation (+, -, *, /) containing a NULL operand yields a result of NULL. For example, calculating 12 * salary * commission_pct evaluates to NULL for any employee without an assigned commission percentage.

- **Incompatibility with Standard Operators:**
  Because NULL represents an unknown quantity, expressions such as column = NULL or column <> NULL evaluate to UNKNOWN rather than TRUE or FALSE, returning zero rows.
- **Filtering with IS NULL Syntax:**
  To retrieve records containing missing data, SQL provides the dedicated IS NULL comparison operator.
  Syntax Example:
  SELECT last_name, manager_id FROM employees WHERE manager_id IS NULL;
- **Filtering with IS NOT NULL Syntax:**
  Conversely, the IS NOT NULL operator restricts query results to records where a specific column contains confirmed, non-missing data.
  Syntax Example:
  SELECT last_name, commission_pct FROM employees WHERE commission_pct IS NOT NULL;

Q7. Detail the usage of column aliases, literal character strings, the concatenation operator (||), and the alternative quote (q) operator in SQL query design.

Introduction: SQL provides flexible formatting mechanisms to customize output column headings and concatenate dynamic table data with static descriptive text literals directly within the SELECT list.

- **Column Aliases (AS Keyword):**
  Column aliases rename output headers to create self-explanatory column labels or format calculated expressions. Using double quotation marks around an alias preserves spaces, special characters, and exact character casing.
  Syntax Example: SELECT salary * 12 AS "Annual Salary" FROM employees;
- **Literal Character Strings:**
  A character string literal is freeform text inserted directly into a SELECT statement list, enclosed in single quotation marks. The static string prints once for every row retrieved by the query.
  Syntax Example: SELECT last_name || ' is an Employee' FROM employees;
- **Concatenation Operator (||):**
  The double vertical bar (||) concatenates multiple text strings or column values together into a single composite character string field in the result set.
  Syntax Example: SELECT first_name || ' ' || last_name AS "Full Name" FROM employees;
- **Alternative Quote (q) Operator:**
  When character literals contain natural single quotes (e.g., apostrophes like "it's" or "Department's"), standard syntax breaks. The q operator defines custom delimiters (such as brackets []) to handle single quotes cleanly without syntax errors.
  Syntax Example: SELECT department_name || q'[, it's assigned Manager ID:]' || manager_id FROM departments;
- **Output Display Integration:**
  Combining aliases, literal strings, and concatenation allows developers to transform raw column values into human-readable narrative text reports directly from SQL.

Q8. Explain row sorting using the ORDER BY clause, including ascending and descending order, multiple column sorting, alias sorting, and NULLS LAST handling.

Introduction: The ORDER BY clause sorts query output rows in a specified sequence. It is always evaluated as the final clause in a SELECT statement.

- **Ascending and Descending Sort Sequences:**
  Sorting defaults to ascending order (ASC), arranging numbers from low to high and text alphabetically. Specifying the DESC keyword reverses the sequence.
  Syntax Example: SELECT * FROM employees ORDER BY hire_date DESC;
- **Sorting by Column Aliases:**
  Because the ORDER BY clause executes after column expressions are evaluated in the SELECT list, calculated column aliases can be referenced directly within ORDER BY.
  Syntax Example: SELECT salary * 12 AS annsal FROM employees ORDER BY annsal ASC;
- **Sorting across Multiple Columns:**
  Queries can sort across multiple columns by listing them in a comma-separated order. Rows are sorted primary by the first column, and ties are resolved by subsequent columns.
  Syntax Example: SELECT * FROM employees ORDER BY department_id ASC, salary DESC;
- **Sorting by Unselected Columns:**
  Data can be sorted using columns that are not included in the SELECT projection list.
- **Explicit Null Positioning (NULLS LAST):**
  By default, null values sort first in descending order and last in ascending order. Adding options like NULLS LAST explicitly forces missing values to the bottom of sorted outputs regardless of sort direction.
  Syntax Example: SELECT * FROM employees ORDER BY last_name ASC NULLS LAST;

Q9. Compare and contrast the WHERE clause and the HAVING clause in SQL using a comparative table and illustrative query examples.

Introduction: Both WHERE and HAVING clauses filter dataset records, but they operate at different stages of query execution and serve distinct logical purposes.

Comparison Feature	WHERE Clause	HAVING Clause
Primary Function	Filters individual source rows before aggregation.	Filters aggregated summary groups after aggregation.
Aggregate Functions	Strictly forbids aggregate functions (e.g., WHERE SUM(sal) > 50).	Explicitly designed to evaluate aggregate functions.
Clause Order	Precedes the GROUP BY clause.	Executed after the GROUP BY clause.
Row vs Group Focus	Operates on raw database table records.	Operates on computed summary groups.

- **WHERE Clause Purpose:**
  The WHERE clause evaluates criteria against raw input table rows individually prior to any grouping operations.
- **HAVING Clause Purpose:**
  The HAVING clause evaluates criteria against computed aggregate group statistics produced by a GROUP BY clause.

- **WHERE Clause Code Example:**
  Filters out employees earning under 50,000 before counting department headcount.
  SELECT department_id, COUNT(*) FROM employees WHERE salary > 50000 GROUP BY department_id;
- **HAVING Clause Code Example:**
  Computes department headcount first, then excludes groups that do not have more than 10 employees.
  SELECT department_id, COUNT(*) FROM employees GROUP BY department_id HAVING COUNT(*) > 10;
- **Combined Execution Context:**
  A single SQL query can utilize both clauses simultaneously to filter individual rows first via WHERE and subsequently filter aggregated groups via HAVING.

Q10. Compare INNER JOIN and OUTER JOIN operations using a comparison table and SQL execution syntax.

Introduction: SQL joins merge records across normalized relational tables based on matching key criteria. INNER JOIN and OUTER JOIN differ fundamentally in how they process unmatched records.

Join Characteristic	INNER JOIN	OUTER JOIN (LEFT, RIGHT, FULL)
Unmatched Rows	Discards unmatched rows from both tables completely.	Retains unmatched rows from one or both tables.
Output Row Count	Returns only exact matching record pairs.	Returns matching pairs plus non-matching records padded with NULLs.
Subtypes	Single standard inner join operation.	Categorized into LEFT OUTER, RIGHT OUTER, and FULL OUTER joins.
Use Case	Strict relational matching between entities.	Preserving master entity lists while retrieving optional related data.

- **INNER JOIN Mechanics:**
  An INNER JOIN evaluates key columns and returns rows only when a matching key value exists in both tables. Unmatched records are omitted from the output.
- **OUTER JOIN Mechanics:**
  An OUTER JOIN preserves records from one or both tables regardless of whether a matching record exists in the joining table. Unmatched attributes fill with NULL values.
- **INNER JOIN Syntax Example:**
  SELECT employees.last_name, departments.department_name FROM employees INNER JOIN departments ON employees.department_id = departments.department_id;
- **OUTER JOIN Syntax Example:**
  SELECT employees.last_name, departments.department_name FROM employees LEFT OUTER JOIN departments ON employees.department_id = departments.department_id;
- **Operational Selection Rule:**
  Use INNER JOIN when requiring strictly linked transactional records; use OUTER JOIN when generating reports that must show all primary entities regardless of secondary table linkage.

Q11. Explain the WITH CHECK OPTION clause in SQL views, detailing its syntax, function, and importance in maintaining underlying table integrity during data modifications.

Introduction: The WITH CHECK OPTION clause is an integrity enforcement setting applied to updatable views to ensure data modifications made through the view comply with the view's logical definition.

- When a view is updatable, users can insert or update rows in the underlying base table via the view. WITH CHECK OPTION prevents data modifications that would cause the inserted or updated row to disappear from the view's result set.
- The clause forces the database engine to check every INSERT or UPDATE executed through the view against the view's defining WHERE clause condition. If the new data violates the WHERE condition, the database rejects the command and throws an error.

Standard Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition
WITH CHECK OPTION;
```

Practical Code Example:

```
CREATE VIEW low_id_students AS
SELECT name, address
FROM student_details
WHERE student_id < 5
WITH CHECK OPTION;
```

Integrity Maintenance Benefits: Without this option, a user could insert a record with student_id = 10 through low_id_students. The insert would succeed in student_details, but the user would not be able to see the record in the view. WITH CHECK OPTION eliminates this logical discrepancy.

Q12. Differentiate between Primary Key and Foreign Key constraints using a comparison table and relational table definitions.

Introduction: Primary Key and Foreign Key constraints are essential relational integrity rules that define identity and establish links across database tables.

Constraint Property	Primary Key Constraint	Foreign Key Constraint
Primary Role	Uniquely identifies each unique record within a table.	Establishes and preserves a link to a Primary Key in another table.
Null Value Allowance	Strictly forbids NULL values; every record must have a key.	Permits NULL values unless constrained by an explicit NOT NULL.
Table Quantity Limit	A table can contain only one Primary Key constraint.	A table can contain multiple Foreign Key constraints.
Uniqueness Rule	Every key entry must be strictly unique across all rows.	Foreign key entries can contain duplicate values across multiple rows.

- **Primary Key Definition:** A Primary Key uniquely identifies each record in a database table, enforcing entity integrity by disallowing duplicates and null entries.
- **Foreign Key Definition:** A Foreign Key is a column or combination of columns that references a Primary Key in another table, enforcing referential integrity between related entities.

  Table Creation Syntax Example:

```
CREATE TABLE departments (
   department_id NUMBER PRIMARY KEY,
   department_name VARCHAR2(50) NOT NULL
);
```

  Referential Link Syntax Example:

```
CREATE TABLE employees (
   employee_id NUMBER PRIMARY KEY,
   last_name VARCHAR2(50) NOT NULL,
   department_id NUMBER,
   CONSTRAINT fk_dept FOREIGN KEY (department_id) REFERENCES
departments(department_id)
);
```

- **Referential Integrity Enforcement:** Foreign Key constraints prevent users from inserting orphan records with nonexistent foreign key values or deleting parent records that ha=ve dependent child rows.

Q13. Compare ANSI standard SQL Statements with Oracle iSQL*Plus Commands using a comparative table and functional capabilities.

Introduction: While ANSI SQL provides the standardized syntax used to manage database data and definitions, iSQL*Plus represents a proprietary environment interface used to execute queries and format reports.

Operational Feature	ANSI Standard SQL Statements	Oracle iSQL*Plus Commands
Core Nature	Standardized relational database language.	Proprietary environment formatting tool.
Database Impact	Directly manipulates database data and table structures.	Formats query outputs and manages script files; cannot modify data.
Keyword Abbreviation	Keywords (SELECT, FROM) cannot be abbreviated.	Environment commands (DESC, EXEC) can be abbreviated.
Execution Architecture	Standardized across all ANSI-compliant RDBMS platforms.	Executed client-side within an Oracle web browser interface.

- **SQL Statement Functionality:**
  SQL statements are issued to the database engine to perform queries, updates, table creations, and structural modifications.
- **iSQL*Plus Command Functionality:**
  iSQL*Plus commands control environment settings, edit script files, load local SQL files, and format displayed query output reports.

- **Interaction Process:**
  Users type SQL queries into the iSQL*Plus workspace. iSQL*Plus sends the SQL statements to the database server and formats the returned query results into web reports.
- **Script Editing Capabilities:**
  iSQL*Plus provides commands to execute code, save workspace code into .sql script files, load existing scripts, and view execution history lists.
- **Display Formatting Controls:**
  Interface options inside iSQL*Plus allow developers to set heading alignment, adjust display page widths, and export execution output into HTML files.

Q14. Explain how subqueries are utilized within the WHERE clause to dynamically filter table data, giving practical multi-table syntax examples.

Introduction: A subquery is a nested SELECT query placed inside an outer SQL statement. When placed in a WHERE clause, subqueries calculate dynamic search conditions based on secondary table evaluations.

- **Dynamic Execution Process:**
  The nested subquery executes first, evaluating its internal query and passing the result set directly to the outer WHERE clause. The main query then filters its records using the dynamic values returned by the subquery.
- **Single-Value vs Multiple-Value Subqueries:**
  Subqueries returning a single value utilize standard comparison operators (=, >), while subqueries returning multiple rows utilize multi-value comparison operators like IN.

Subquery with IN Operator Syntax:

```
SELECT *
FROM employees
WHERE department_id IN (
   SELECT department_id
   FROM departments
   WHERE location = 'New York'
);
```

Explanation: The subquery finds all department_id values located in New York, and the outer query retrieves employees belonging to those departments.

- **Decoupling Hardcoded Values:**
  Using subqueries removes the need to hardcode changing values (such as department IDs or average salaries) into application code, ensuring queries adapt automatically as database data changes.
- **Combining Subqueries with Aggregations:**
  Subqueries inside WHERE clauses frequently calculate dynamic aggregate benchmarks (e.g., retrieving employees earning above the company-wide average salary).
  SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

Section C: 2-Mark Short Solutions

Q15. Define Projection and Selection as foundational capabilities of the SQL SELECT statement.

Projection and Selection represent the two core horizontal and vertical data extraction dimensions of the SQL SELECT statement.

- **Projection:** The operation of selecting specific columns from a table while omitting unrequested attributes.
- **Selection:** The operation of restricting query output rows based on specified logical criteria using the WHERE clause.

Q16. What is the purpose of the DISTINCT keyword in SQL queries? Provide its basic syntax.

By default, SQL displays all rows evaluated by a query, including duplicate output rows.

- **Purpose:** The DISTINCT keyword eliminates duplicate rows from query results, ensuring that only unique values or combinations of values are displayed.
- **Syntax:** SELECT DISTINCT column1, column2 FROM table_name;

Q17. Define the SQL LIKE operator and describe its two wildcard characters.

The LIKE operator performs string pattern matching on character columns using search wildcards.

- **Wildcard % (Percent):** Matches zero, one, or multiple arbitrary characters.
- **Wildcard _ (Underscore):** Matches exactly one single character position.
  Example: WHERE first_name LIKE 'A%' retrieves names starting with 'A'.

Q18. Differentiate between COUNT(*) and COUNT(column_name) in aggregate calculations.

COUNT() is an aggregate function that computes row counts in a dataset.

- **COUNT(*):** Counts all rows in the target table or group, including rows containing NULL values and duplicate records.
- **COUNT(column_name):** Counts only non-null values in the specified column, completely ignoring NULL records.

Q19. Define a CROSS JOIN and state its mathematical relationship to joined table row counts.

A CROSS JOIN produces the complete Cartesian product of two joined relational tables.

- **Definition:** A join operation that pairs every single row from the first table with every single row in the second table.
- **Mathematical Relationship:** If Table 1 contains M rows and Table 2 contains N rows, the resulting query dataset contains exactly M x N total rows.

Q20. Explain the concept and syntax of a SELF JOIN in SQL.

A SELF JOIN is a query operation in which a database table is joined with itself.

- **Concept:** It treats a single table as two distinct logical tables by assigning unique table aliases, enabling comparisons between rows within the same table.
- **Syntax Example:**
  SELECT A.last_name AS Emp, B.last_name AS Mgr FROM employees A, employees B WHERE A.manager_id = B.employee_id;

Q21. Define a View in SQL and state the SQL command used to delete a view.

A View is a virtual database table whose contents are defined by an underlying SELECT query.

- **Definition:** Virtual tables that present rows and columns from underlying base tables without storing physical data records independently.
- **Deletion Command:**
  DROP VIEW view_name;

Q22. How are integrity constraints dropped from existing database tables in SQL? State the command syntax.

Integrity constraints enforce mandatory data validation rules on table columns.

- **Mechanism:** Constraints are removed from existing table structures using the ALTER TABLE statement combined with the DROP CONSTRAINT option.
- **Command Syntax:**
  ALTER TABLE table_name DROP CONSTRAINT constraint_name;